Objects, References, and Encapsulation

CSCI 2113 — Lab 2: building the dispatch board

Alex (Aizierjiang Aiersilan)
Lab Lead, Teaching Assistant

August 30, 2026

The George Washington University
CSCI 2113 - Software Engineering | Fall 2026

© Aizierjiang Aiersilan. Free distribution and open-access.

Outline

1. From Lab 1 to Lab 2
2. Part 1 — Defining a class
3. Part 2 — Constructors and `this`
4. Part 3 — References & memory
5. Part 4 — `toString`
6. Part 5 — Encapsulation
7. Part 6 — `equals`
8. The assignment
9. Running the lab
10. Acknowledgement

ng Alersilan | CSCI 2113

From Lab 1 to Lab 2

ng Aiersilan | CSCI 2113 | F

Where Lab 1 left us

Lab 1: one driver was spread across **four parallel arrays**.

```
String[]  names;
int[]     zones;
boolean[] available;
double[]  ratings;
```

- ✗ Index 3 is "Ken" in four places at once.
- ✗ Sort one array, corrupt the record.
- ✗ Add a field → touch every method.

Lab 2: one driver is **one object**.

```
Driver ken =
    new Driver("Ken", 1, true, 4.6);
```

- ✓ The four values travel together.
- ✓ The object can answer questions.
- ✓ Adding a field is a local change.

💡 Same data. Better shape.

What students should be able to do by the end

1. Define a **class** with fields, a **constructor**, and methods.
2. Create objects with `new` and use them through **references**.
3. Explain **reference semantics**: ==, **aliasing**, and where objects live.
4. Compile and run a program made of **more than one file**.
5. Apply **encapsulation**: `private` fields, get/set methods, validation at construction time.
6. Implement `toString()` and `equals()`.

The one sentence for the whole lab

A class bundles data and the code that guards it into a thing you can hand around by **reference**.

How today runs

Six short activities, then the assignment.

1. Define a class — *fields, defaults*
2. Constructor and `this`
3. References and `==`
4. `toString()`
5. `private`, getters, validation
6. `equals()`

Everything builds one file: **Driver.java**.

The rule for every activity

★ Predict the output. Write it down.
Then run it.

Two files, one command

```
javac Driver.java DispatchBoard.java
java DispatchBoard
```

ng Aiersilan / CSCI 2113

Part 1 — Defining a class

ng Aiersilan | CSCI 2113 | F

A class is a blueprint, not a thing

Driver.java — type this exactly:

```
public class Driver {
    String name;
    int zone;
    boolean available;
    double rating;
}
```

No constructor. No methods. Just a **shape**: four **instance fields** that every **Driver** will have.

Vocabulary to say out loud

Class — the blueprint, written once.

Object — one thing built from it, at run time.

Field — a named slot inside each object.

⚠ **File rule:** a `public` class must live in a file of exactly the same name, capital letter included.

Two files, one program

DispatchBoard.java — the program that uses it:

```
public class DispatchBoard {
    public static void main(String[] args) {
        Driver d = new Driver();
        System.out.println(d.name);
        System.out.println(d.zone);
        System.out.println(d.available);
        System.out.println(d.rating);
    }
}
```

```
$ javac Driver.java DispatchBoard.java
$ java DispatchBoard
```

`javac` produces **two** `.class` files — one per class, not one per program.

`java` wants a **class name** here:

✔ `java DispatchBoard`

✘ `java DispatchBoard.java`

(`java DispatchBoard.java` *does* run on a modern JDK — Java 11+ compiles one source file in memory, and Java 22+ follows the reference into **Driver.java** too [2, 6]. It leaves no `.class` files behind, and the handout wants you to see those — so use `javac` here.)

What does a brand-new object hold?

Nobody assigned anything. Java still guarantees a value for every **instance field** [3].

Field	Type	Prints
`name`	`String`	**null**
`zone`	`int`	**0**
`available`	`boolean`	**false**
`rating`	`double`	**0.0**

The two ideas here

Zero of the right kind. Numbers get **0**, booleans **false**, and every **reference** type gets **null**.

Fields differ from local variables. *Reading* a local variable you never assigned is a compile error. A field is filled in for you.

⚠ `rating` prints **0.0**, not **0** — the `double` is telling you its type.

Reaching fields through a reference

```
d.name = "Ada";
d.zone = 3;
d.available = true;
d.rating = 4.8;
System.out.println(d.name + " | zone "
        + d.zone + " | " + d.rating);
```

```
Ada | zone 3 | 4.8
```

Read `d.name` out loud

"the `name` field *of the object that* `d` *refers to*."

`d` is not the object. `d` is a **reference** — an arrow pointing at one.

⚠ Two problems we will fix today: nothing stops `d.rating = 99.0;`, and every caller must know the field names.

Part 2 — Constructors and `this`

ng Aiersilan | CSCI 2113 | F

Activity 2: write the constructor

```
public class Driver {
    String name;
    int zone;
    boolean available;
    double rating;

    Driver(String name, int zone,
           boolean available, double rating) {
        this.name = name;
        this.zone = zone;
        this.available = available;
        this.rating = rating;
    }
}
```

Three rules for a constructor

1. Same name as the class, exactly.
2. **No return type** — not even `void`.
3. Runs once, when `new` builds the object.

`this` means *the object being built right now*. It is how you say "the field", not "the parameter" [5].

The no-argument constructor disappears

Recompile without touching **DispatchBoard.java**:

```
DispatchBoard.java:3: error: constructor Driver in class Driver cannot be applied to given types;
        Driver d = new Driver();
                   ^
  required: String,int,boolean,double
  found:    no arguments
  reason: actual and formal argument lists differ in length
```

`new Driver()` worked a minute ago because a class with **no constructor at all** gets a **default constructor** for free [3]. Write **one** constructor and that gift is withdrawn.

The fix

```
Driver ada = new Driver("Ada", 3,
true, 4.8);
```

That is the language enforcing **valid at birth**.

The silent bug: leaving out `this`

```
Driver(String name, int zone, ...) {
    name = name;    // no this
    zone = zone;
    ...
}
```

```
$ javac -Xlint:all Driver.java DispatchBoard.java
$ java DispatchBoard
null | zone 0 | 0.0
```

Why this is worth five minutes

It compiles. It runs. Not one warning — even with `-Xlint:all`.

Inside the constructor, the bare word `name` means the **parameter**, because the nearest declaration wins. So `name = name;` assigns the parameter to itself and the field is never touched — it keeps its default.

✔ `this.name = name;` reads: field ← parameter.

Part 3 — References & memory

ng Aiersilan | CSCI 2113 | F

Activity 3: predict, then run

```
Driver a = new Driver("Grace", 1, false, 5.0);
Driver b = new Driver("Grace", 1, false, 5.0);
Driver c = a;

System.out.println(a == b);   // ?
System.out.println(a == c);   // ?
c.zone = 2;
System.out.println(a.zone);   // ?
```

Before anyone runs it

Everyone writes **three** answers on paper. Hands up for each one.

Then two questions:

- How many **Driver** objects exist?
- What do `a`, `b`, `c` each *hold*?

Where the objects actually live

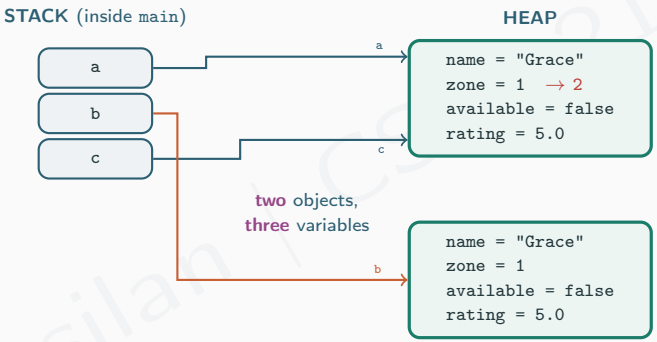


Figure 1: a and c hold the *same* arrow; b holds a different one. Changing c.zone changes what a sees.

The three answers

Expression	Prints	Because
`a == b`	**false**	`==` compares the **arrows**, not the fields. Two `new` calls, two objects.
`a == c`	**true**	`c = a` copied the arrow. One object, two names.
`a.zone` *(after* `c.zone = 2`*)*	**2**	`a` and `c` are the same object, so the write is visible through both.

For objects, == asks "same object?" — never "same contents?"

Aliasing: the sentence to say out loud

The rule

Assigning an object variable copies the reference, never the object.

Two variables pointing at one object are **aliases**. A write through either is seen by both — there is no copy to fall out of date.

Same rule as Lab 1's `board[2]` experiment: an array element holds a reference too, one level down.

Where this bites later

- Passing a **Driver** to a method and mutating it → the caller sees the change.
- Two array slots holding one **Driver** → one edit, two rows change.

⚠ `int` and `double` do *not* behave this way: those copy the **value**.

Part 4 — `toString`

ng Aiersilan | CSCI 2113 | F

Before `toString`: what `println(a)` prints

```
System.out.println(a);
```

```
Driver@2a139a55
```

⚠ The digits are an **identity hash code** — not an address, not a field. They say nothing about Grace, and they may differ on another machine or JDK. Do not chase them.

What just happened

`println` needs text, so it calls the object's `toString()`. **Driver** does not have one, so it **inherits** the version every object gets: the class name, an `@`, and `hashCode()` in hexadecimal.

That is Java saying: "I have no idea how you want this printed." So tell it.

Writing `toString`

```
public String toString() {
    return this.name + " | zone " + this.zone
                     + " | " + this.rating;
}
```

```
Grace | zone 1 | 5.0
```

Now `println(a)`, `"" + a` and string concatenation all use it. You never call `toString()` by hand.

The handout's target line, `Grace | zone 1 | 5.0`, pads the name — same content, next slide for the spacing.

⚠ If your `println(a)` sits *after* `c.zone = 2`, you will see `zone 2`. That is aliasing, not a bug.

The signature must match

`public String toString()`

`public` · returns `String` · named exactly `toString` · **no parameters**.

⚠ Change any one of those and you have written a *different* method. `println` will keep printing `Driver@2a139a55` [1].

Making the columns line up

```
public String toString() {
    return String.format("%-8s | zone %d | %.1f",
        this.name, this.zone, this.rating);
}
```

```
Ada      | zone 3 | 4.8
Grace    | zone 1 | 5.0
Barbara  | zone 2 | 4.9
```

`Grace    | zone 1 | 5.0` is exactly the handout's Activity 4 line.

`%-8s` — text, left-aligned, padded to **8** columns.
`%d` — an `int`.
`%.1f` — a `double`, **1** decimal place.

⚠ Width **8** covers the longest name, **Barbara** (**7** letters), so every bar lands in the same column. Padding never truncates: a longer name just pushes its bar right.

⚠ The handout's assignment sample uses a narrower pad (`%-6s |`). Pick *one* width, use it everywhere, and follow your grader if they specify one.

ng Aiersilan | CSCI 2113

Part 5 — Encapsulation

ng Aiersilan | CSCI 2113 | F

`private`: the compiler stops you

Mark all four fields `private`, recompile *before* adding any get methods. With the Activity 3 `main` still in place:

```
DispatchBoard.java:8: error: zone has private access in Driver
        c.zone = 2;
         ^
DispatchBoard.java:9: error: zone has private access in Driver
        System.out.println(a.zone);   // ???
                           ^
2 errors
```

`private` means visible only inside this class. Not the package, not a subclass — this class [5].

⚠ `c.zone = 2` is a *write*, and there will be no `setZone` — delete that line; its aliasing job is done.

Why break working code on purpose

Every one of those errors is a place that was reaching **past** the object instead of **asking** it. Now you can see them all.

Get methods, and the naming rule

```
public String getName()      { return this.name; }
public int getZone()         { return this.zone; }
public boolean isAvailable() { return this.available; }
public double getRating()    { return this.rating; }
```

Then fix **DispatchBoard**: `a.zone` becomes `a.getZone()`, and `c.zone = 2` goes. Every *read* of a field becomes a question.

The convention

`getX()` for most types.

`isX()` for a `boolean` — `isAvailable()`, not `getAvailable()`.

It is only a convention — the JavaBeans naming rules, not the language [4] — but every Java tool and every reader expects it. Follow it.

Aiersilan | CSU 2113

`setRating`: validation in one place

```
public void setRating(double rating) {
    if (rating < 0.0) {
        this.rating = 0.0;
    } else if (rating > 5.0) {
        this.rating = 5.0;
    } else {
        this.rating = rating;
    }
}
```

Given	Stored
-3.0	0.0
9.9	5.0
4.25	4.25

💡 This is what a `private` field buys you: one doorway, so the rule cannot be dodged.

Validate at construction too

```
Driver(String name, int zone,
       boolean available, double rating) {
    this.name = name;
    this.zone = zone;
    this.available = available;
    // not this.rating = rating;
    this.setRating(rating);
}
```

```
Driver k = new Driver("Ken", 1, true, -3.0);
k.getRating()  ->  0.0
k.setRating(9.9);
k.getRating()  ->  5.0
```

The invariant

A **Driver**'s rating is always between **0.0** and **5.0** — from the instant it exists, for its whole life.

Assigning the field directly in the constructor leaves a **one**-statement hole in the rule. Calling the setter closes it.

⚠ Ask the room: *is* `-3.0` *a bad rating, or a caller bug?* Clamping hides it; throwing would report it. Today we clamp.

What encapsulation actually bought us

Before

- ✗ `d.rating = 99.0;` compiles.
- ✗ Every caller knows the field names.
- ✗ Rename a field, break the world.

After

- ✓ The only way in is `setRating`.
- ✓ Callers know **questions**, not storage.
- ✓ Change the inside freely.

The mental shift

Stop asking an object *for its data* so you can compute. Ask it a question and let it answer. **Dispatch** will never touch a **Driver** field — and it will not need to.

Part 6 — `equals`

ng Aiersilan | CSCI 2113 | F

Activity 6: == versus `equals`

```
Driver a = new Driver("Grace", 1, false, 5.0);
Driver b = new Driver("Grace", 1, false, 5.0);

System.out.println(a == b);         // ?
System.out.println(a.equals(b));    // ?
```

```
false
false
```

The surprise

Most students predict **false** then **true**.
Let them be wrong —
it is a better lesson than being told.

With no `equals` of its own, **Driver** inherits the one every object has — and that one **compares references**. Identical to ==.

Writing `equals`

```
public boolean equals(Driver other) {
    return this.name.equals(other.name)
        && this.zone == other.zone;
}
```

```
a == b       ->  false
a.equals(b)  ->  true
```

You are deciding what **equal** *means* for a **Driver**: same name, same zone. Rating and availability do not count — that is a design decision, and it is yours to make.

⚠ `other.name` is legal here. `private` is **per class**, not per object — inside **Driver** you can see any **Driver**'s fields.

The overload trap (know this, even if they do not)

```
Object o = b;                    // same object as b
System.out.println(a.equals(b));  // true
System.out.println(a.equals(o));  // false
```

The inherited method is `equals(Object)`. Writing `equals(Driver)` adds a **second** method beside it — an **overload**, not an **override**. Java picks by the declared type of the argument, at compile time.

Why it matters downstream

`ArrayList.contains`, `HashMap`, `Arrays.equals` all call `equals(Object)`. They will not see yours.

Today's handout asks for `equals(Driver)` — fine, and it works for the tests you will run. The real-world version takes `Object`, checks the type, casts, and is marked `@Override` [1].

ng Aiersilan / Cornell 2113

Never compare Strings with ==

```
String typed = new String("Grace");  // built at run time

"Grace" == "Gra" + "ce"      ->  true   // both literals
"Grace" == typed             ->  false
"Grace".equals(typed)        ->  true
```

`String` is an object, so == still asks **"same object?"**
Literals (and constant expressions like `"Gra" + "ce"`) are **interned** — one shared instance per spelling — so == *appears* to work [3].

The trap

Their code passes with hard-coded names, then fails the moment a name comes from `Scanner`, a file, or concatenation.

✔ `a.equals(b)` ✘ `a == b`

The assignment

ng Aiersilan | CSCI 2113 | F

Rebuild the board — with objects this time

Deliverables

- **Driver.java** — copied from the activity: `private` fields, constructor, get methods, `setRating`, `toString`, `equals`.
- **Dispatch.java** — the given data plus **4** `static` methods.

The four methods

1. `printBoard()`
2. `countAvailable()`
3. `bestRatedAvailable()`
4. `nearestAvailable(int riderZone)`

The one hard constraint

Driver's fields stay `private`. **Dispatch** reaches driver data only through get methods.

If a method would be easier with a public field, that is the exercise working. Add a **question** to **Driver** instead.

The data you are given

```
static Driver[] drivers = {
    new Driver("Ada",     3, true,  4.8),
    new Driver("Grace",   1, false, 5.0),
    new Driver("Linus",   4, true,  4.1),
    new Driver("Ken",     1, true,  4.6),
    new Driver("Barbara", 2, false, 4.9),
    new Driver("Alan",    4, true,  4.4)
};
```

6 drivers, **4** available (Ada, Linus, Ken, Alan).

`distance[a][b]` = minutes from zone `a` to zone `b`.

	0	**1**	**2**	**3**	**4**
0	0	5	9	12	14
1	5	0	6	8	11
2	9	6	0	4	7
3	12	8	4	0	5
4	14	11	7	5	0

⚠ Zones run **0–4**, but no driver sits in zone **0**. Riders can.

printBoard() and countAvailable()

```
public static void printBoard() {
    for (int i = 0; i < drivers.length; i++) {
        System.out.println(drivers[i]);
    }
}
```

```
Ada    | zone 3 | 4.8 | AVAILABLE
Grace  | zone 1 | 5.0 | UNAVAILABLE
```

The handout's sample, verbatim (`%-6s |`); keep the width you chose in Part 4.

`println(drivers[i])` calls `toString()` for you — so **Dispatch** needs no getters at all here. Extend `toString()` to append `AVAILABLE` or `UNAVAILABLE`.

`countAvailable()` returns **4**: one counter, one loop, `isAvailable()` as the test.

⚠ `drivers.length` — no parentheses. Arrays have a **field**; `String` has a `length()` **method**.

bestRatedAvailable(): the best-so-far pattern

```
public static Driver bestRatedAvailable() {
    Driver best = null;
    for (int i = 0; i < drivers.length; i++) {
        Driver d = drivers[i];
        if (!d.isAvailable()) continue;
        if (best == null
            || d.getRating() > best.getRating()) {
            best = d;
        }
    }
    return best;     // null if nobody is available
}
```

Returns **Ada** (**4.8**) — not **Grace** (**5.0**), who is unavailable.

Two habits worth naming

Start with `null`, never with `drivers[0]` — driver zero may be unavailable.

Return the object, not the name. The caller can print it or ask it anything else.

`nearestAvailable(int riderZone)`: two rules at once

```
Driver best = null;
int bestDist = 0;
for (int i = 0; i < drivers.length; i++) {
    Driver d = drivers[i];
    if (!d.isAvailable()) continue;
    int dist = distance[riderZone][d.getZone()];

    if (best == null || dist < bestDist) {
        best = d;  bestDist = dist;          // rule 1
    } else if (dist == bestDist
            && d.getRating() > best.getRating()) {
        best = d;  bestDist = dist;          // rule 2
    }
}
return best;
```

1. Smallest `distance[riderZone][driverZone]`.
2. Tie → higher rating wins.
3. Nobody available → **null**.

⚠ `bestDist` is only ever read *after* `best` is non-null, so its starting value never matters. Say that out loud — it is the part students get wrong.

The answers, for checking work at the bench

Rider zone	Answer	Minutes		Why
0	Ken	5	(zone 1)	Ada 12, Linus and Alan 14
1	Ken	0	(zone 1)	same zone as the rider
2	Ada	4	(zone 3)	Ken 6, Linus and Alan 7
3	Ada	0	(zone 3)	same zone as the rider
4	Alan	0	(zone 4)	tie with Linus at 0 → rating 4.4 beats 4.1

Zone 4 is the only case that exercises the tie-break. If someone's zone 4 answers Linus, rule 2 is missing.

Running the lab

ng Aiersilan | CSCI 2113 | F

The four compile errors you will be asked about

Message	What the student actually did
`constructor Driver in class Driver cannot be applied to given types`	Added a constructor, still calling `new Driver()`.
`name has private access in Driver`	**Dispatch** touched `d.name` instead of `d.getName()`.
`cannot find symbol ... method setRating(double)`	Constructor calls `this.setRating(...)` but the method is not written yet.
`class Driver is public, should be declared in a file named Driver.java`	Filename and class name disagree, usually by one capital letter.

Teach the habit: read the first error only, fix it, recompile. Later errors are often echoes of the first.

Bugs that compile but lie

- ✗ `name = name;` → prints `null | zone 0 | 0.0`. No warning, ever.
- ✗ Strict `<` with no tie-break → zone **4** returns **Linus** (**4.1**) instead of **Alan** (**4.4**).
- ✗ `int bestDist = 0;` with only `dist < bestDist` → returns **null** for every zone.
- ✗ `name == other.name` → passes with literals, fails with typed input.
- ✗ Ignoring a **null** return → `NullPointerException`.
- ✗ `distance[5][...]` → `ArrayIndexOutOfBoundsException: Index 5 out of bounds for length 5`.

⚠ One to mention only if asked: `0.1 + 0.2 == 0.3` is **false** — never compare doubles with `==`.

Checkpoints: what to ask at each bench

Understanding, not output

1. "How many objects does this code make?"
2. "Point at the arrow that `c` holds."
3. "Why did `new Driver()` stop compiling?"
4. "What does `this` refer to on this line?"

Late in the session

5. "Make `setRating` reject **-3.0** — where is the one place you edit?"
6. "Which zone proves your tie-break works?"
7. "`printBoard` calls no getter — so where does each line's text come from?"

💡 A student who can answer **2** and **7** has the whole lab — one is references, the other is delegation.

Running code is not the same as understanding it.

Takeaways

1. ★ **A variable holds an arrow, not an object** — so == asks "same object?" and assignment creates an **alias**.
2. ★ **Constructors make objects valid at birth** — and `this` is how you tell a field from a parameter.
3. ★ `private` **turns data into questions** — one doorway means the rule cannot be dodged, and **Dispatch** never needs to look inside.

In one sentence

Lab 1 had **four** arrays that happened to line up; Lab 2 has one object that keeps its own promises.

Acknowledgement

ng Aiersilan | CSCI 2113 | F

Acknowledgement

This lab builds on the initial materials developed by **Timothy Kim**.

We gratefully acknowledge his contribution.

ng Aiersilan | CSCI 2113

References

[1] J. Bloch. ***Effective Java.*** Addison-Wesley, 3rd edition, 2018. Item 10 (`equals` contract), Item 12 (always override `toString`), Item 40 (`@Override`).

[2] J. Gibbons. **JEP 330: Launch Single-File Source-Code Programs.** OpenJDK, `https://openjdk.org/jeps/330`, 2018. Delivered in JDK 11.

[3] J. Gosling, B. Joy, G. Steele, G. Bracha, A. Buckley, D. Smith, and G. Bierman. ***The Java Language Specification, Java SE 21 Edition.*** Oracle America, Inc., Aug. 2023. Sect. 4.12.5 (Initial Values of Variables), Sect. 8.8.9 (Default Constructor), Sect. 3.10.5 (String Literals). `https://docs.oracle.com/javase/specs/jls/se21/html/`.

[4] G. Hamilton, editor. ***JavaBeans API Specification, Version 1.01.*** Sun Microsystems, Inc., July 1997. Sect. 8.3.1–8.3.2: `get<Property>()` / `set<Property>()`, and `is<Property>()` for boolean properties. `https://www.oracle.com/java/technologies/javase/javabeans-spec.html`.

[5] Oracle Corporation. **The Java Tutorials: Lesson: Classes and Objects.** `https://docs.oracle.com/javase/tutorial/java/javaOO/`, 2024. Trail *Learning the Java Language*; pages *Providing Constructors for Your Classes*, *Using the this Keyword* and *Controlling Access to Members of a Class*. Written for JDK 8; accessed 2026-08-29.

[6] R. Pressler. **JEP 458: Launch Multi-File Source-Code Programs.** OpenJDK, `https://openjdk.org/jeps/458`, 2024. Delivered in JDK 22.

Let's write some code.

Questions?

Backup: `Driver.java` in its final form

```java
public class Driver {
    private String name;
    private int zone;
    private boolean available;
    private double rating;

    Driver(String name, int zone,
           boolean available, double rating) {
        this.name = name;
        this.zone = zone;
        this.available = available;
        this.setRating(rating);   // validate at birth
    }

    public String getName()      { return this.name; }
    public int getZone()         { return this.zone; }
    public boolean isAvailable() { return this.available; }
    public double getRating()    { return this.rating; }

    public void setRating(double rating) {
        if (rating < 0.0)      this.rating = 0.0;
        else if (rating > 5.0) this.rating = 5.0;
        else                   this.rating = rating;
    }

    public String toString() {
        return String.format("%-8s | zone %d | %.1f | %s",
            this.name, this.zone, this.rating,
            this.available ? "AVAILABLE" : "UNAVAILABLE");
    }

    public boolean equals(Driver other) {
        return this.name.equals(other.name)
            && this.zone == other.zone;
    }
}
```

⚠ The constructor has no access modifier because the handout writes it that way; `public` is the better habit — both compile in the default package. Its **4** parameters match every `new Driver(...)` in the given **Dispatch.java**.

Backup: the two loop shapes that solve everything

Count / accumulate

```
int count = 0;
for (int i = 0; i < drivers.length; i++) {
    if (drivers[i].isAvailable()) {
        count++;
    }
}
return count;
```

Best-so-far / search

```
Driver best = null;
for (int i = 0; i < drivers.length; i++) {
    Driver d = drivers[i];
    if (/* d fails a filter */ false) continue;
    if (best == null
        || /* d beats best */ false) {
        best = d;
    }
}
return best;
```

💡 `countAvailable` is the left one. `bestRatedAvailable` and `nearestAvailable` are both the right one — only the filter and the "beats" test change.